



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY	3
CLIENT.....	3
INSTRUCTIONS.....	3
DEVELOPER	4
1. ALGORITHM CIPHER	4
2. CERTIFICATE GENERATION	4
3. DEPLOY CIPHER.....	4
4. SECURE COMMUNICATIONS	4
5. SECONDARY TESTING.....	4
6. FUNCTIONAL TESTING	4
7. SUMMARY	4
8. INDUSTRY STANDARD BEST PRACTICES.....	4

Document Revision History

Version	Date	Author	Comments
1.0	06/25/2026	Cindi Rodriguez	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer

Cindi Rodriguez

1. Algorithm Cipher

For Artemis Financial, I recommend using AES-256 (Advanced Encryption Standard) because it provides strong protection for sensitive financial information. Since the company stores and transfers personal and financial data, it is important to use an encryption algorithm that is both secure and widely trusted. AES is the current industry standard and is used by banks, healthcare organizations, and government agencies because it offers a high level of security while still performing efficiently.

This project also uses the SHA-256 hashing algorithm but hashing and encryption have different purposes. Encryption protects data by converting it into a format that can only be read with the correct key. Hashing on the other hand creates a unique digital fingerprint of the data. In this project, SHA-256 was used to generate a checksum so the application can verify that the data has not been changed or tampered with. SHA-256 produces a 256-bit hash value, while AES-256 uses a 256-bit encryption key to protect sensitive information.

AES is a symmetric encryption algorithm, which means the same key is used to both encrypt and decrypt the data. This makes it much faster than asymmetric encryption when protecting large amounts of information. Asymmetric encryption uses two different keys. A public key and a private key and is commonly used for secure communication through HTTPS and digital certificates. Randomly generated keys are also important because they make encrypted data much harder for attackers to predict or break.

Encryption has improved significantly over the years as technology has advanced. Older algorithms like DES are no longer considered secure because their key sizes are too small to protect against modern attacks. AES replaced DES and has become the standard because it supports stronger key sizes of 128, 192, and 256 bits while remaining efficient and resistant to known attacks. For Artemis Financial, combining AES-256, SHA-256 hashing and HTTPS creates multiple layers of security that help protect customer information and verify that data remains accurate during transmission.

2. Certificate Generation

Insert a screenshot below of the CER file.

I generated a self-signed certificate using Java Keytool. The certificate was created with a 2048-bit RSA key and exported as a CER file named server.cer. This certificate supports HTTPS communication in the application during testing and development. The certificate printout confirms the owner, issuer, validity dates, fingerprint information, and public key algorithm.

```

Terminal Shell Edit View Window Help

find: /Users/cindirodriguezflores/Library/StatusKit: Operation not permitted
find: /Users/cindirodriguezflores/Library/DoNotDisturb: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.HomeKit: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/CloudKit: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.Safari: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.findmy.imagecache: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.findmy.fmcore: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.containernavigator: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/FamilyCircle: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.homed: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.findmy.fmipcore: Operation not permitted
find: /Users/cindirodriguezflores/Library/Caches/com.apple.ap.adprivacyd: Operation not permitted
find: /Users/cindirodriguezflores/.Trash: Operation not permitted
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads % find ~/Downloads -name "*.cer"
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads % find ~/eclipse-workspace -name "*.cer"
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads % cd ~/Downloads
keytool -genkeypair -keyalg RSA -alias selfsigned -keypass ChangeMe123! -keystore keystore.jks -storepass ChangeMe123! -validity 360 -keysize 2048
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
[Unknown]: Cindi Rodriguez
What is the name of your organizational unit?
[Unknown]: IT Department
What is the name of your organization?
[Unknown]: Artemis Financial
What is the name of your City or Locality?
[Unknown]: Mapleton
What is the name of your State or Province?
[Unknown]: Utah
What is the two-letter country code for this unit?
[Unknown]: US
Is CN=Cindi Rodriguez, OU=IT Department, O=Artemis Financial, L=Mapleton, ST=Utah, C=US correct?
[no]: yes

Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 360 days
for: CN=Cindi Rodriguez, OU=IT Department, O=Artemis Financial, L=Mapleton, ST=Utah, C=US
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads % keytool -export -alias selfsigned -storepass ChangeMe123! -file server.cer -keystore keystore.jks
Certificate stored in file <server.cer>
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads % keytool -printcert -file server.cer
Owner: CN=Cindi Rodriguez, OU=IT Department, O=Artemis Financial, L=Mapleton, ST=Utah, C=US
Issuer: CN=Cindi Rodriguez, OU=IT Department, O=Artemis Financial, L=Mapleton, ST=Utah, C=US
Serial number: dc98196cfe2213fa
Valid from: Fri Jun 26 06:16:50 MDT 2026 until: Mon Jun 21 06:16:50 MDT 2027
Certificate fingerprints:
    SHA1: 98:65:DE:96:BE:F4:59:D4:F3:7F:DD:C6:38:1D:72:F5:F6:2A:11:37
    SHA256: 77:E1:2B:98:DB:8B:2A:85:6F:77:1A:24:6F:75:42:32:22:F6:7C:EC:6A:4C:8D:89:BB:5F:C7:D9:B9:6F:DE:59
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

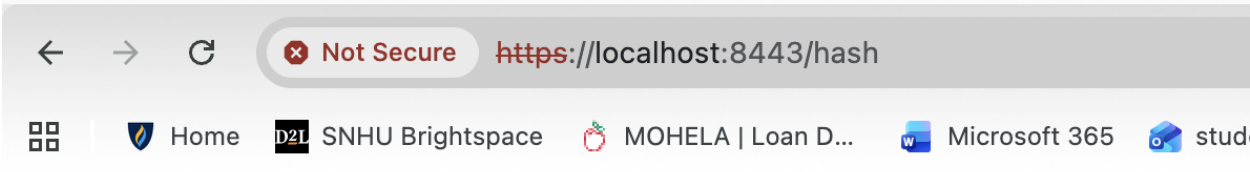
Extensions:
#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: F0 88 A9 72 FF 3A AE D5 7B 27 3E 0C 67 11 81 1A ...r:...'.>.g...
0010: 19 D1 89 88
]
]
cindirodriguezflores@Cindis-MacBook-Air-2 Downloads %
```

3. Deploy Cipher

Insert a screenshot below of the checksum verification.

I refactored the code to implement the SHA-256 cryptographic hash algorithm. I added a new /hash endpoint to the Spring Boot application that generates a checksum for a unique string. This checksum verifies the integrity of the data by producing a unique hash value. If the data changes, the checksum also changes, making it possible to detect tampering.

The screenshot below shows the application successfully generating a SHA-256 checksum for my unique data string.



Data: Cindi Rodriguez

Algorithm: SHA-256

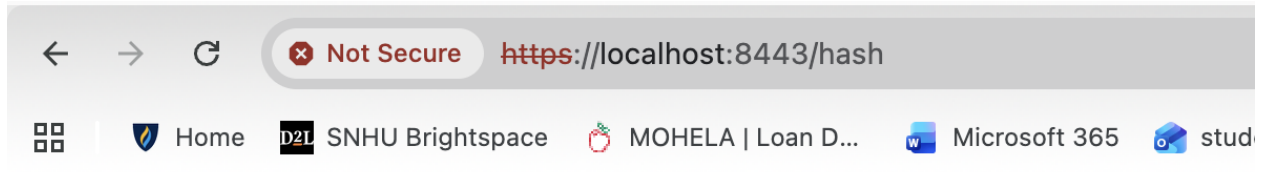
Checksum: 0d82cfc9a9c9f50c17e0312233accb14d68af6974673d1cacfc7629e3fdb317d

4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.

I configured the application to use HTTPS instead of HTTP by updating the application properties file and using a self-assigned certificate generated with the Java Keytool. After running the application, I verified that secure communication was working by accessing the application through

<https://localhost:8443/hash>. Although the browser displays a “Not Secure” warning because the certificate is self-assigned, the application is successfully using HTTPS to encrypt communication between the client and server.



Data: Cindi Rodriguez

Algorithm: SHA-256

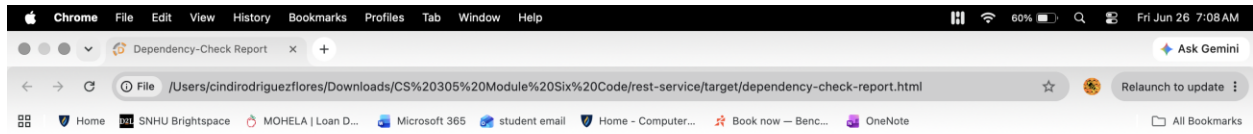
Checksum: 0d82cfc9a9c9f50c17e0312233accb14d68af6974673d1cacfc7629e3fdb317d

5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.

After refactoring the application, I ran the OWASP Dependency-Check tool again to perform a secondary static security scan. The purpose of this scan was to verify that the security improvements I made, including implementing the SHA-256 checksum and configuring HTTPS, did not introduce any new vulnerabilities.

The application was compiled and executed successfully, confirming that the refactored code functioned correctly. The Dependency-Check report showed that one known vulnerability was successfully suppressed using the suppression file I created. Although the report still identified vulnerabilities within third party dependencies, these were existing library vulnerabilities and were not introduced by the code I added. Overall, the secondary testing confirmed that my security enhancements complied with the required software security testing protocols.

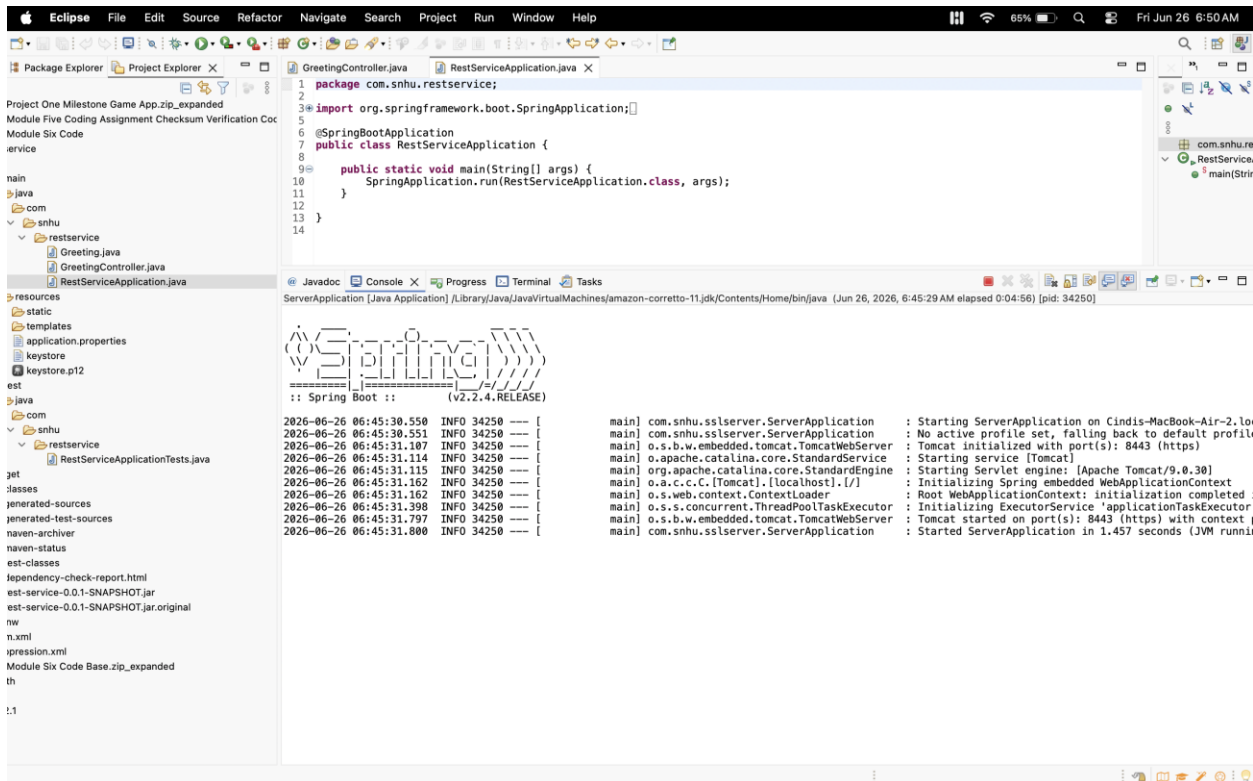


Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncycastle-crypto-package:1.46:*:*:*:* cpe:2.3:a:bouncycastle:bouncycastle_for_java:1.46:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncycastle-java-cryptography-api:1.46:*:*:* cpe:2.3:a:bouncycastle:the_bouncycastle_crypto_package_for_java:1.46:*:*:*	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	3	Highest	38
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:* cpe:2.3:a:hibernate-validator:hibernate-validator:6.0.18:*:*:	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	4	Highest	32

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncycastle-crypto-package:1.46:*:*:* cpe:2.3:a:bouncycastle:bouncycastle_for_java:1.46:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncycastle-java-cryptography-api:1.46:*:*: cpe:2.3:a:bouncycastle:the_bouncycastle_crypto_package_for_java:1.46:*:*:	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	3	Highest	38
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*: cpe:2.3:a:hibernate-validator:hibernate-validator:6.0.18:*:*:	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	4	Highest	32



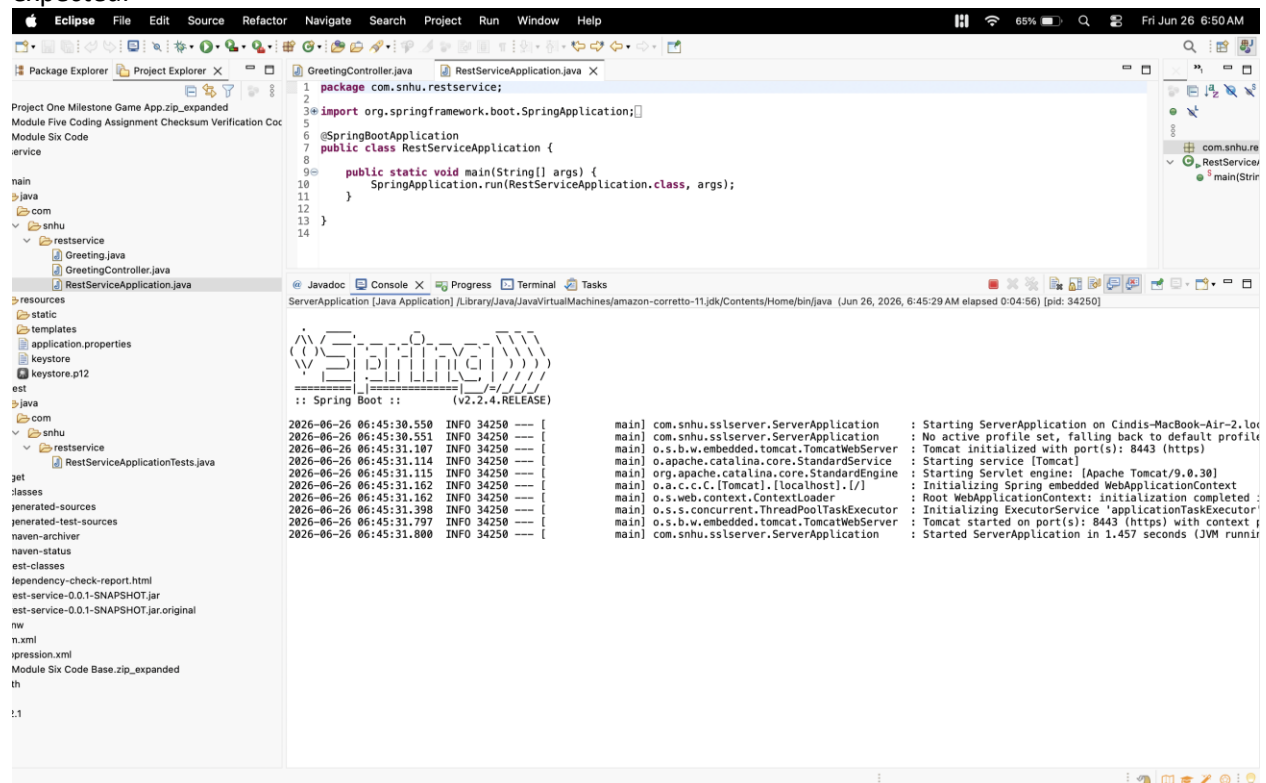
6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.

After refactoring the application, I manually reviewed the code to identify any syntactical, logical, and security issues before running the program.

First, I checked for syntax errors to make sure the project complied successfully without any build errors. I then reviewed the application logic to confirm that the /hash endpoint correctly generated a SHA-256 checksum for my unique data string. Finally, I verified the security of the application by confirming that HTTPS was enabled, the self-assigned certificate was working correctly, and the checksum was generated using the SHA-256 algorithm.

After reviewing the code, I ran the application in Eclipse and verified that it executed successfully without any errors. I also tested the application by opening the /hash endpoint in my browser to confirm that the checksum was generated correctly and that secure communication was functioning as expected.



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, including the 'restservice' package and its sub-packages. The main editor window shows the 'RestServiceApplication.java' file, which contains the following code:

```
1 package com.snhu.restservice;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class RestServiceApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(RestServiceApplication.class, args);
11     }
12 }
13
14
```

The console output at the bottom shows the application running successfully. The output includes the following lines:

```
2026-06-26 06:45:30.550 INFO 34250 --- [main] com.snhu.sslserver.ServerApplication : Starting ServerApplication on Cindis-MacBook-Air-2.10.11.11
2026-06-26 06:45:30.551 INFO 34250 --- [main] com.snhu.sslserver.ServerApplication : No active profile set, falling back to default profile
2026-06-26 06:45:31.107 INFO 34250 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8443 (https)
2026-06-26 06:45:31.114 INFO 34250 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-26 06:45:31.115 INFO 34250 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-06-26 06:45:31.162 INFO 34250 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-26 06:45:31.162 INFO 34250 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed
2026-06-26 06:45:31.398 INFO 34250 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2026-06-26 06:45:31.797 INFO 34250 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8443 (https) with context
2026-06-26 06:45:31.800 INFO 34250 --- [main] com.snhu.sslserver.ServerApplication : Started ServerApplication in 1.457 seconds (JVM running)
```

7. Summary

Throughout this project, I refactored the original application to improve its security and make it better align with secure software development practices. I implemented a SHA-256 cryptographic algorithm to generate checksums, which helps verify that data has not been changed during transmission. I also configured the application to use HTTPS with a self-signed certificate so that communication between the client and server is encrypted.

As I worked through the project, I followed several steps to improve the application's security. I generated and exported a self-signed certificate, configured the application to use HTTPS instead of HTTP, implemented the checksum functionality, manually tested the application, and ran the OWASP Dependency-Check tool to identify potential vulnerabilities. After reviewing the Dependency-Check report, I confirmed that the security changes I made did not introduce any new vulnerabilities into the application.

The vulnerability assessment process helped me address several important areas of security, including secure communication, encryption, certificate management, data integrity, and vulnerability testing. By combining encryption, secure communication, manual code reviews, and automated security testing, I added multiple layers of protection to the application instead of relying on just one security feature. This layered approach makes the application more secure and helps reduce the risk of data being intercepted or modified.

8. Industry Standard Best Practices

Throughout this project, I applied several industry standard best practices to improve and maintain the security of the application. Instead of creating my own encryption method, I used the SHA-256 hashing algorithm, which is a widely accepted standard for protecting data integrity. I also configured the application to use HTTPS with a self-signed certificate so that data transmitted between the client and server is encrypted. To verify that my changes did not introduce new security issues, I manually tested the application and used the OWASP Dependency-Check tool to scan for known vulnerabilities. These practices helped maintain the application's existing security while adding additional layers of protection.

Using industry standard best practices is important because it helps organizations reduce security risks, protect sensitive customer information, and build trust with their users. For Artemis Financial, implementing secure communication, encryption, and vulnerability testing helps protect financial and personal data from unauthorized access to tampering. Following established security practices also makes it easier to identify and fix vulnerabilities before the software is deployed, reducing the likelihood of security incidents and helping the company maintain a secure and reliable application.